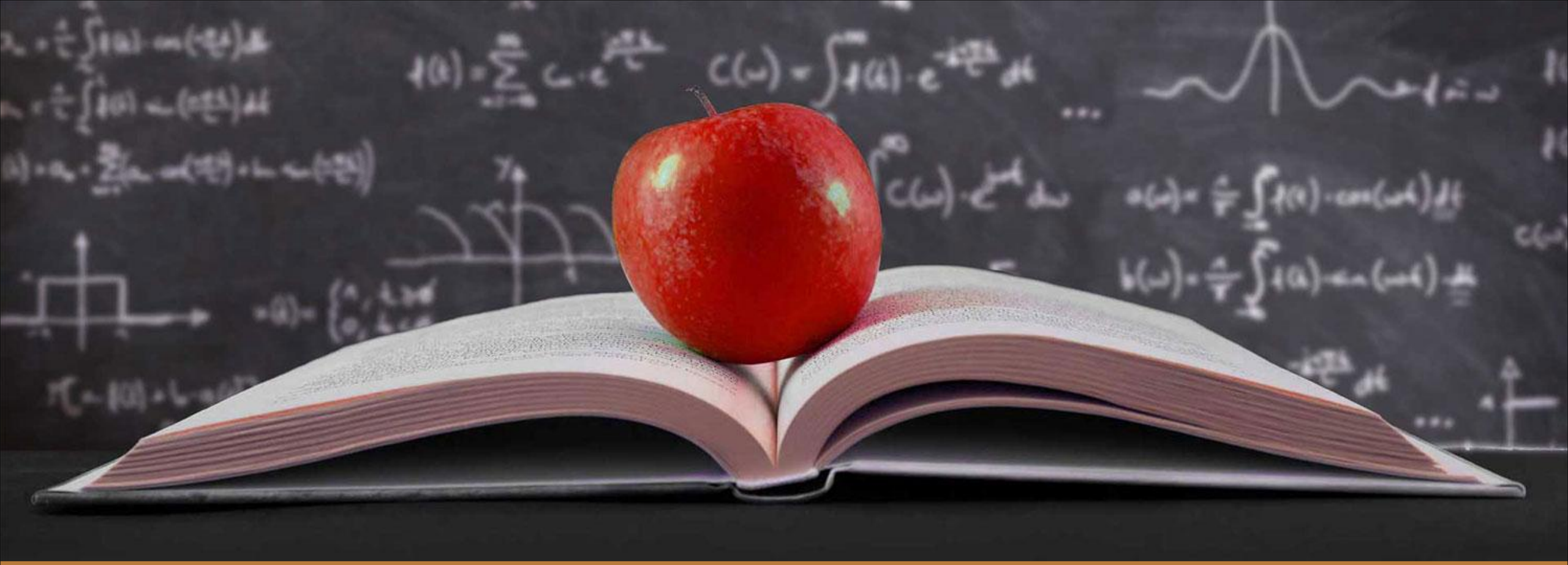
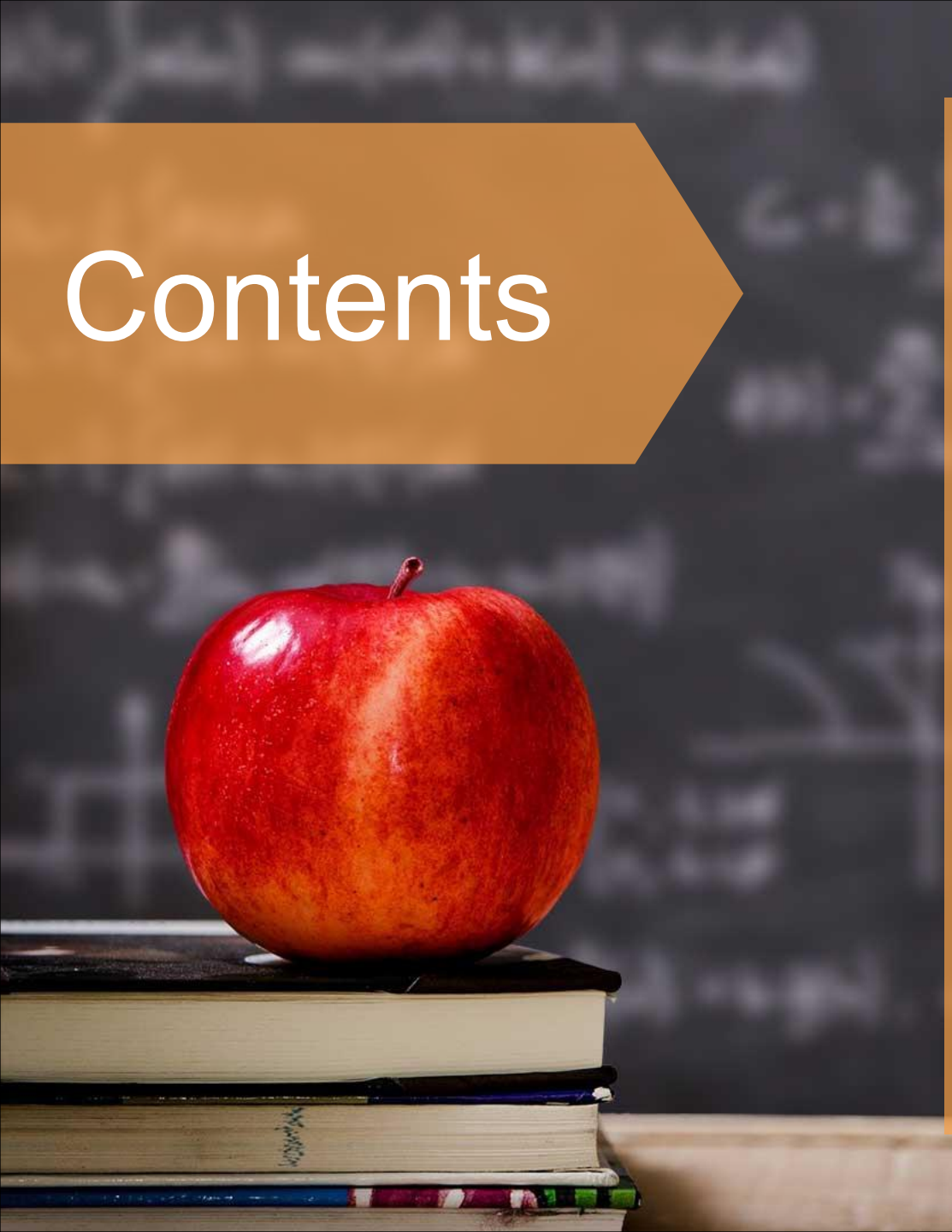


Array Techniques

Contents



01 Introduction

02 Array Order Reversal

03 Array Counting

04 Finding the Maximum number in a set

Introduction



- An array is a special variable, which can hold more than one value at a time.
- If you have a list of items (a list of car names, for example), storing the cars in single variables could look like this:

  car1 = "Ford"

  car2 = "Volvo"

  car3 = "BMW“
- An array can hold many values under a single name, and you can access the values by referring to an index number.

Access the Elements of an Array

- An array element by referring to the *index number*.

Example

- Get the value of the first array item:

```
x = cars[0]
```

- Modify the value of the first array item:

```
cars[0] = "Toyota“
```

- Use the len() method to return the length of an array (the number of elements in an array).
- Return the number of elements in the cars array:

```
x = len(cars)
```

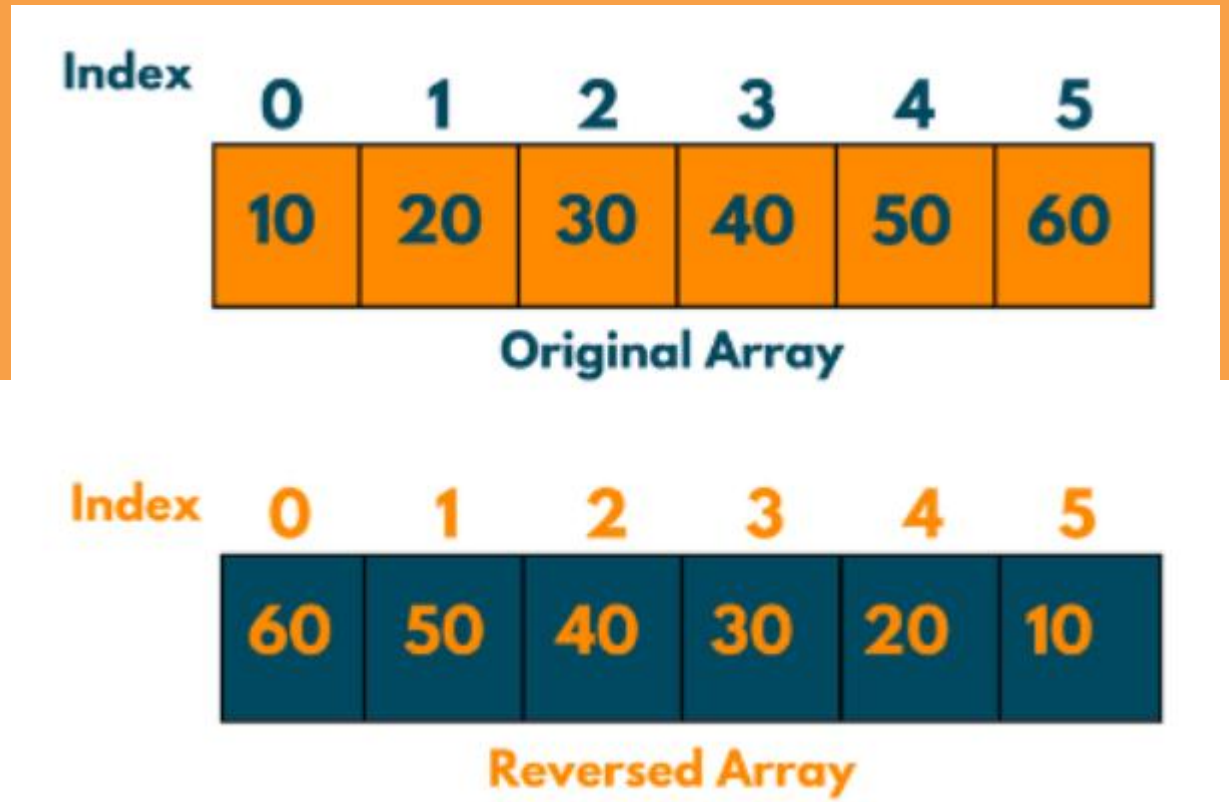
Array Methods

Python has a set of built-in methods that can be used on lists/arrays.



Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

Reverse Array in Python



Declare an Array



- The array module has to be imported in order to use arrays.

```
#import the array module
import array as arr
#initializing array with integers
a = arr.array('i', [1,2,3,4,5])
print('The array is: ',a)
```

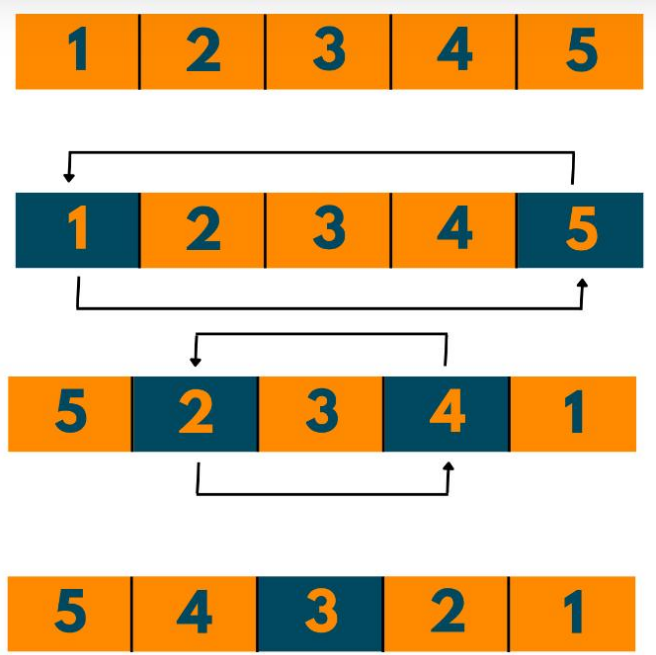
In order to declare an array - declare the data type it is storing

Output:

```
The array is: array('i', [1, 2, 3, 4, 5])
```

Method: 1 Swapping the elements

```
#import the array module
import array as arr
# initializing array with integers
a = arr.array("i", [1, 2, 3, 4, 5])
print("The array: ", a)
# get size of the array using len() method
l = len(a)
# going through the array only halfway!!
for i in range(l // 2):
# swap the elements
a[i], a[l - i - 1] = a[l - i - 1], a[i]
print("The reversed array: ", a)
```



Output:

The array: array('i', [1, 2, 3, 4, 5])

The reversed array: array('i', [5, 4, 3, 2, 1])

Method:2 Using reverse() Method



- To reverse an array in Python you can use the reverse() method directly on both, lists and arrays.
- Using the reverse() method is beneficial when you have a memory space constraint since it performs the operation on the same memory and doesn't require any extra space.
- This is one of the fastest methods to reverse an array in Python.
- This method is also suitable for long-length arrays.

Contd...



```
#import the array module
import array as arr
# initializing array with integers
a = arr.array("i", [1, 2, 3, 4, 5])
print("The array: ", a)
#calling reverse() method on array a
a.reverse()
print("The reversed array: ", a)
```

Output:

```
The array: array('i', [1, 2, 3, 4, 5])
The reversed array: array('i', [5, 4, 3, 2, 1])
```

Method:3 Using reversed() Method



- To reverse an array, Python also offers reversed() method.
- The reversed() method doesn't create any copy of the array, nor does it make any changes in the original array. Instead, it returns an iterator.
- This method is made to facilitate reverse iteration over sequences.

Contd...



```
# import the array module
import array as arr
# initializing array with integers
a = arr.array("i", [1, 2, 3, 4, 5])
print("The array: ", a)
#passing the array in reversed()
reversed(a)
print("The reversed array: ", a)
```

Output:

```
The array: array('i', [1, 2, 3, 4, 5])
The reversed array: array('i', [5, 4, 3, 2, 1])
```

Contd...



```
# import the array module
import array as arr
# initializing array with integers
a = arr.array("i", [1, 2, 3, 4, 5])
print("The array: ", a)
#passing the array in reversed()
reversed_a = reversed(a)
print("The reversed array: ", reversed_a)
```

Output:

```
The array: array('i', [1, 2, 3, 4, 5])
 The reversed array: <reversed object at
0x0000025EC428EC88>
```

Contd...



- The reversed() method returned an iterator, which got stored and printed further.
- In order to get a reversed list, i.e. a copy of the reverse of the original array, you need to apply list() over the reversed_a variable.
- Indirectly, convert the iterator to a list in order to get the reversed array.

Contd...



```
# import the array module
import array as arr
# initializing array with integers
a = arr.array("i", [1, 2, 3, 4, 5])
print("The array: ", a)
#passing the array in reversed()
reversed_a = list (reversed(a))
print("The reversed array: ", reversed_a)
```

Output:

```
The array: array('i', [1, 2, 3, 4, 5])
The reversed array: array('i', [5, 4, 3, 2, 1])
```

Contd...



- Although the reverse() and reversed() perform the same function, they are different.
- While reverse() works on the same array, without consuming any extra memory space, reversed() needs a new array to store a copy of the reversed array.
- Also, while the reverse() just reverses the original array, reversed() does not affect the original array and returns an iterator, to reverse iterate over a sequence.
- reversed() returns an iterator, it only works with sequential objects. The reversed() method will not be applicable to sets and iterators.

Method:4 Using Slicing



- This is the most basic and used method for reversing an array.
- It has no effect on the original array; instead, it returns a reversed copy of the original array.
- The Slicing method iterates over the array from beginning to end (last index).
- **Syntax:** **arrayName[start: end: step]**

CONTD...



- To reverse an array, the step parameter plays the most important role.
- For example, slice the array a = [1, 2, 3, 4]
  1. from start= 0 to end = 2 => a[:3]
  2. from start = 2 to rest of the array => a[2:]
  3. from start = 1 to rest of the array, skipping the next element
     => a[1::1]
- The syntax of slicing to reverse an array in Python is:
  arrayName[::-1]

Contd...



```
# import the array module
import array
#function to reverse the array
def reversingArray(a):
    return a[::-1]
# declare an array
a = array.array("i", [1, 2, 3, 4, 5])
print('The original array is: ', a)
print('The reversed array is ',reversingArray(a)) reversingArray(a)
#The above statement won't work. Can you guess, why?
print(a)
```

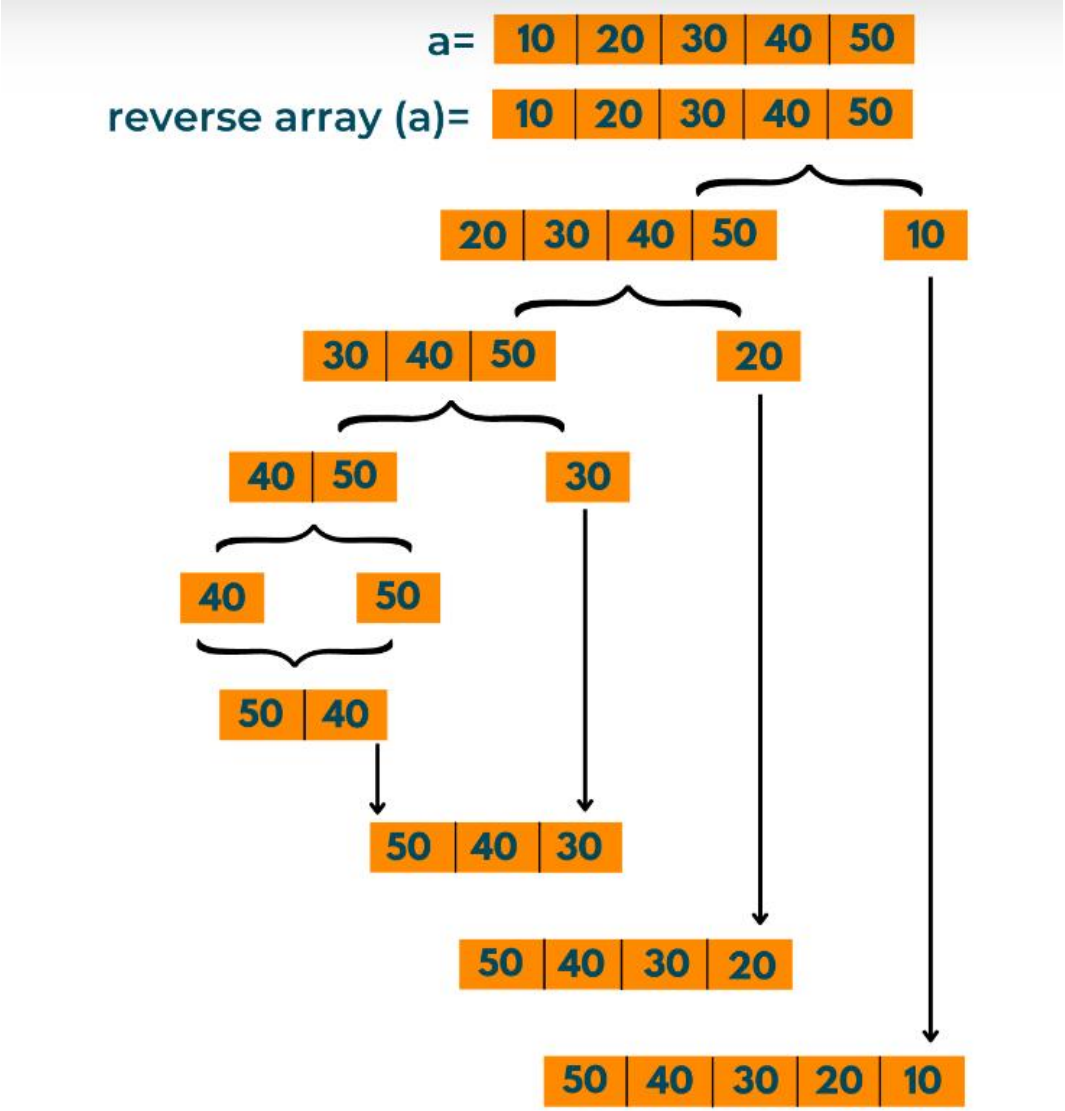
Method:4 Using Slicing



```
# import the array module
import array
# declare an array
a = array.array("i", [1, 2, 3, 4, 5])
print('The original array is: ', a)
#slicing the array
print('The reversed array is ', a[::-1])
#printing the original array again
print('Effect on original array: ', a)
```

```
The original array is: array('i', [1, 2, 3, 4, 5])
The reversed array is array('i', [5, 4, 3, 2, 1])
Effect on original array: array('i', [1, 2, 3, 4, 5])
```

Method:5 Using Recursion



Contd...



- Recursion is also one way to reverse an array in Python, without using any in-built functions.
- When during execution, a function calls for itself, it is known as recursion.
- One needs to be careful while dealing with Recursion since it can lead to an infinite loop or execution until the stack overflows.
- It is necessary to add a terminating condition in the function.

Contd...



```
#import array module
import array
#function to reverse array
def reverseArray(arr):
        if len(arr) == 1:
        return arr
#print(arr) ->You can uncomment this to know how recursion works
return reverseArray(arr[1:]) + arr[0:1]
#declaring array
a = array.array('i', [10, 20, 30, 40, 50])
print("The original array: ", a)
print('The reversed array is: ',reverseArray(a))
```

Contd...



```
#declaring an array with length = 1
b = array.array('i', [1])
print('Original array: ', b)
print('The reversed array: ', reverseArray(b))
```

```
The original array: array('i', [10, 20, 30, 40, 50])
The reversed array is: array('i', [50, 40, 30, 20, 10])
Original array: array('i', [1])
The reversed array: array('i', [1])
```



Array Counting

30.01.2023

Array Counting

len(array)

Finding the Array Length in Python using the len() method



Array module

Finding the Length of a Python Array

NumPy module

Finding the Length of a Python NumPy Array

Array.count(x)

The **count() method** is used to return the number of occurrences of a value or item in an array.

Contd...



```
a = [1,2,3,4,'Python']
print("List elements: ",a)
print("Length of the list:",len(a))
```

Output:

```
List elements:  [1, 2, 3, 4, 'Python']
Length of the list: 5
```

```
import array as A
arr = A.array('i',[1,2,3,4,5])
print("Array elements: ",arr)
print("Length of array:",len(arr))
```

Output:

```
Array elements:  array('i', [1, 2, 3, 4, 5])
Length of array: 5
```

Contd...



```
import numpy as np
arr = np.arange(5)
len_arr = len(arr)
print("Array elements: ",arr)
print("Length of NumPy array:",len_arr)
```

Output:

```
Array elements:  [0 1 2 3 4]
Length of NumPy array: 5
```

Contd...



```
# importing the array module
import array as arr
# creating an integer data type array
x = arr.array('i', [1,2,3,4,5,6,1,2,3,2,4,7])
# using the count function for the value 2 in the array
y = x.count(2)
# printing the number of counts
print('The number of occurrences of the value 2 in the array is: ', y)
```

Contd…

Arrayname = [ele1, ele2, ele3,….]

len() method returns the length of the array in Python.

```
#Initialize array
arr = [1, 2, 3, 4, 5];
#Number of elements present in an array can be found using len()
print("Number of elements present in given array: " + str(len(arr)));
```

Contd...

Count frequencies of all elements in array in Python

Output

1: 3

2: 4

3: 5

```
# intializing the list arr = [1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3,
3]
# initializing dict to store frequency of each element
elements_count = {}
# iterating over the elements for frequency
for element in arr:
 # checking whether it is in the dict or not
if element in elements_count:
# inceremenTing the count by 1
elements_count[element] += 1
        else:
# setting the count to 1
 elements_count[element] = 1
# printing the elements frequencies
for key, value in elements_count.items():
```

CONTD...

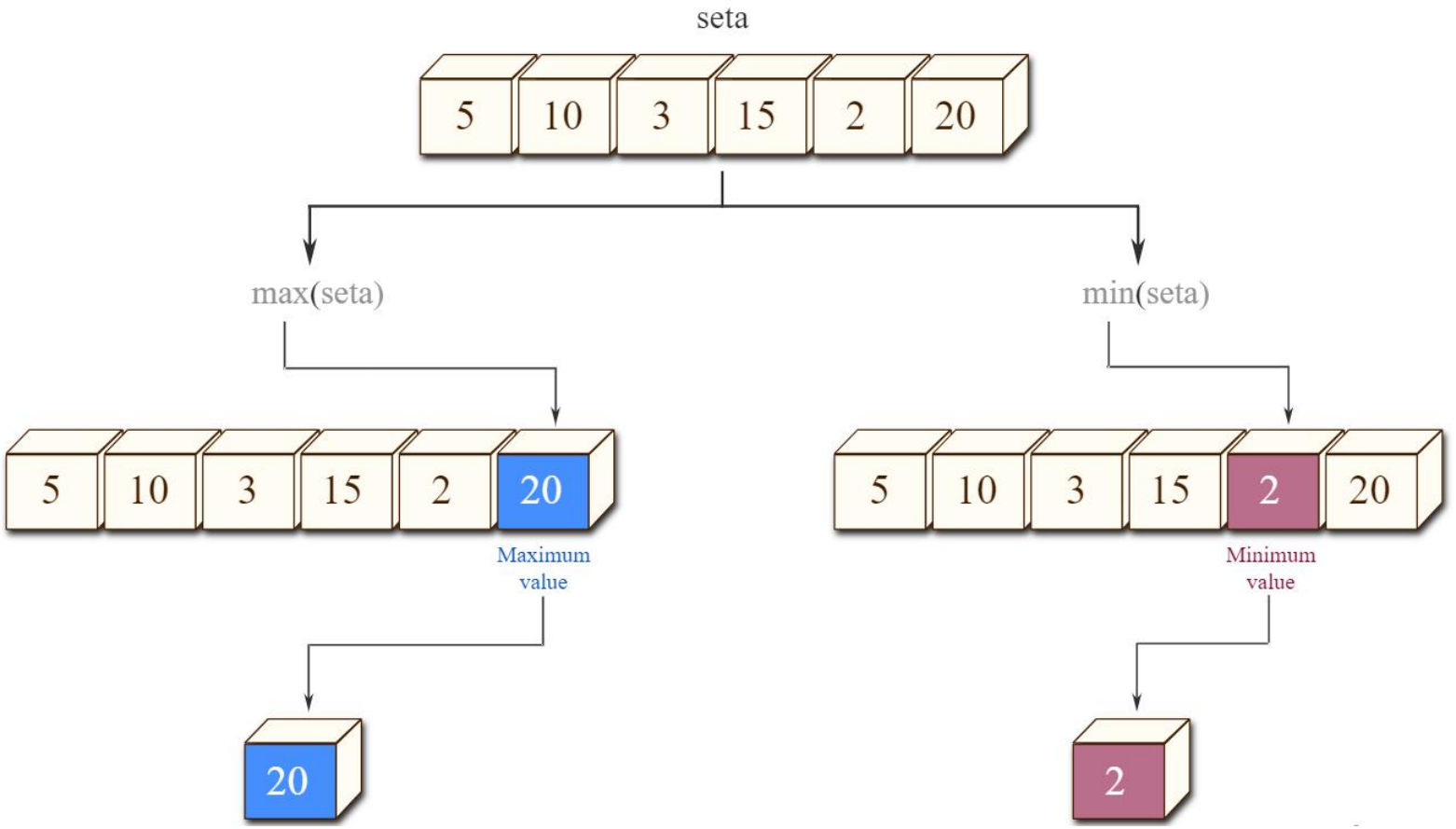


```
# Example Python program that counts the number of occurrences of a specific
# element present in an array
import array
# Create an array of signed integers
numbers = array.array('i');
# Add elements to the array
numbers.append(1);
numbers.append(2);
# Add 3 twice
numbers.append(3);
numbers.append(3);
# Count the number of occurrences of 3
num2Find = 3;
occurences = numbers.count(num2Find);
print("Number of occurrences of %d:"%num2Find);
print(occurences);
```

OUTPUT:

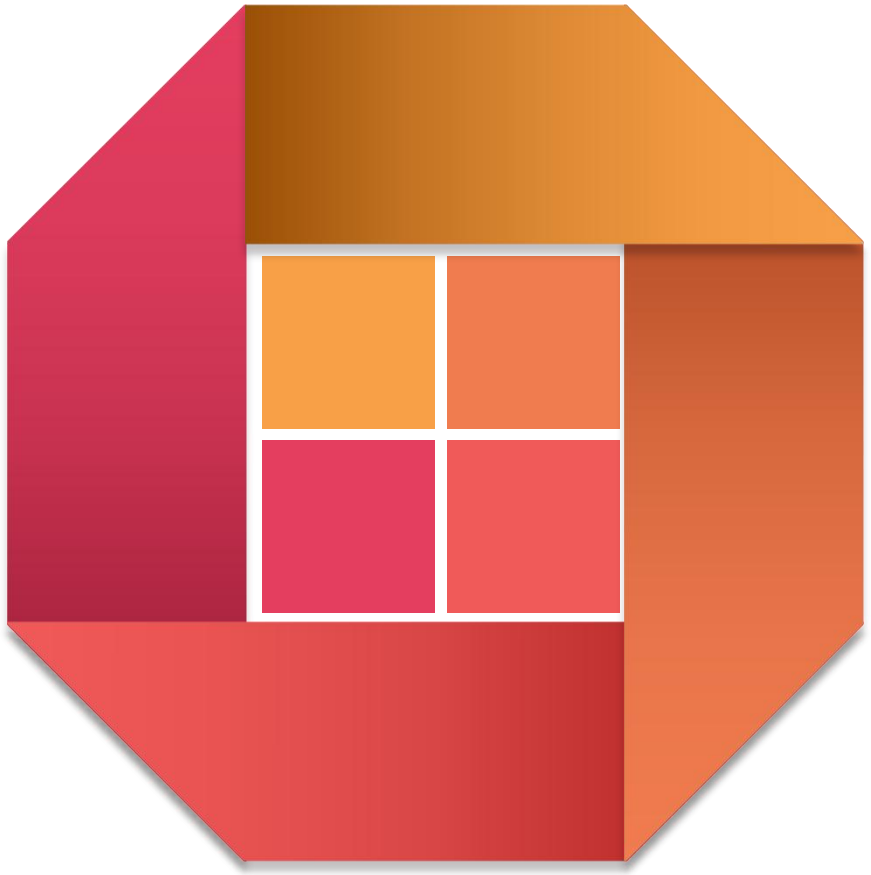
```
Number of occurrences of 3:
2
```

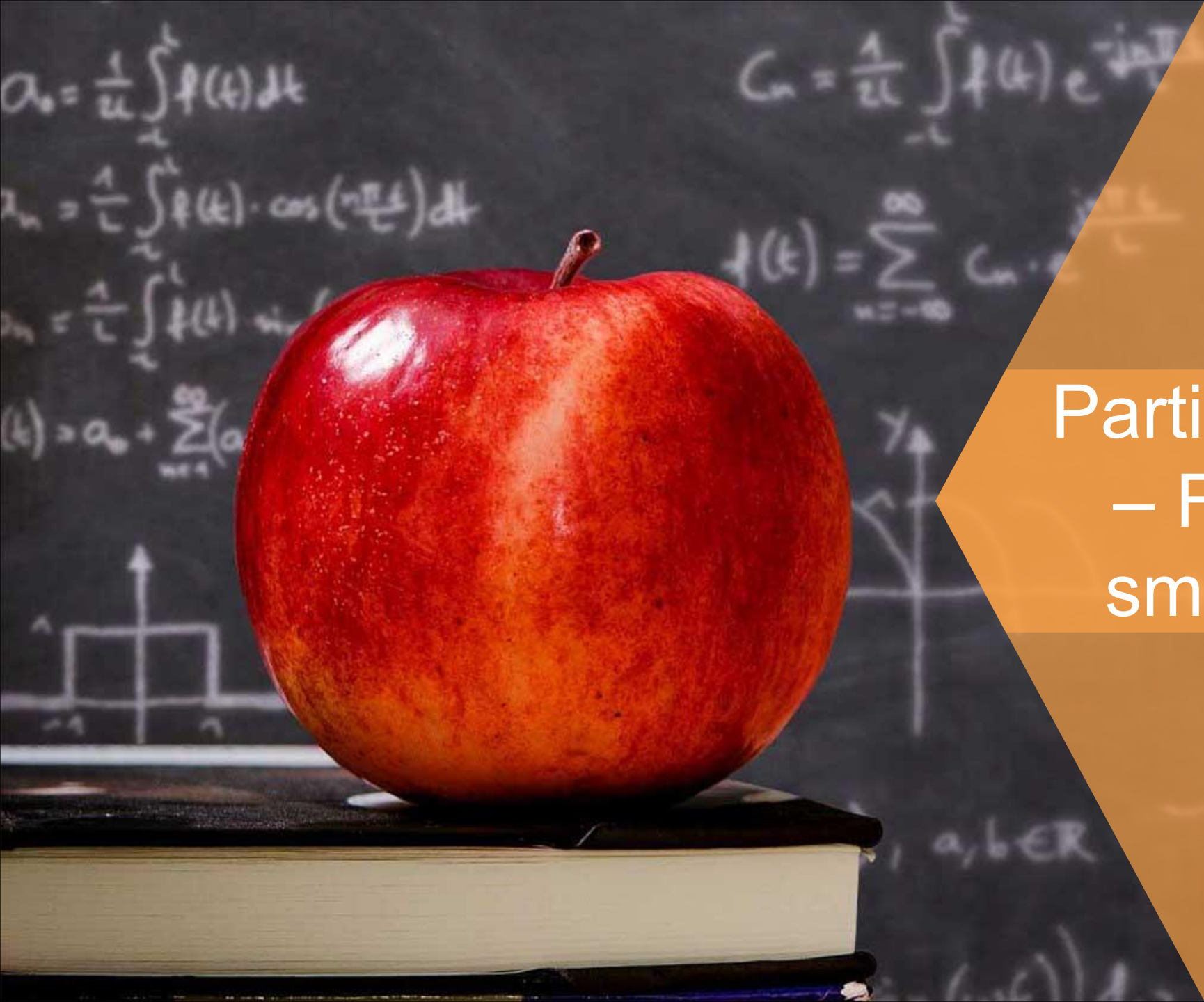
Finding the Maximum number in a set



Program

```
#Create a set setn = {5, 10, 3, 15, 2, 20}
print("Original set elements:")
print(setn)
print(type(setn))
print("\nMaximum value of the said set:")
print(max(setn))
print("\nMinimum value of the said set:")
print(min(setn))
```





Partitioning an array – Finding the Kth smallest element

01.02.2023

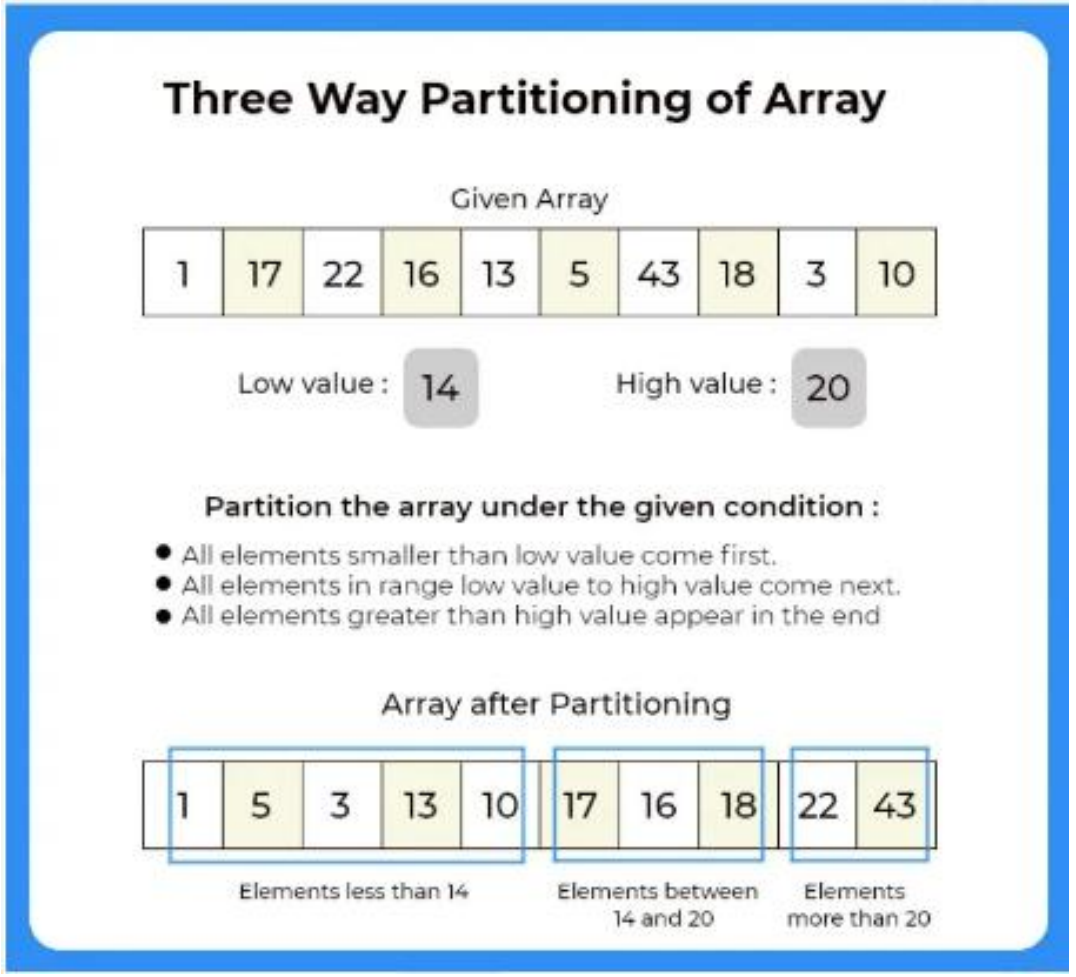
Partitioning an array

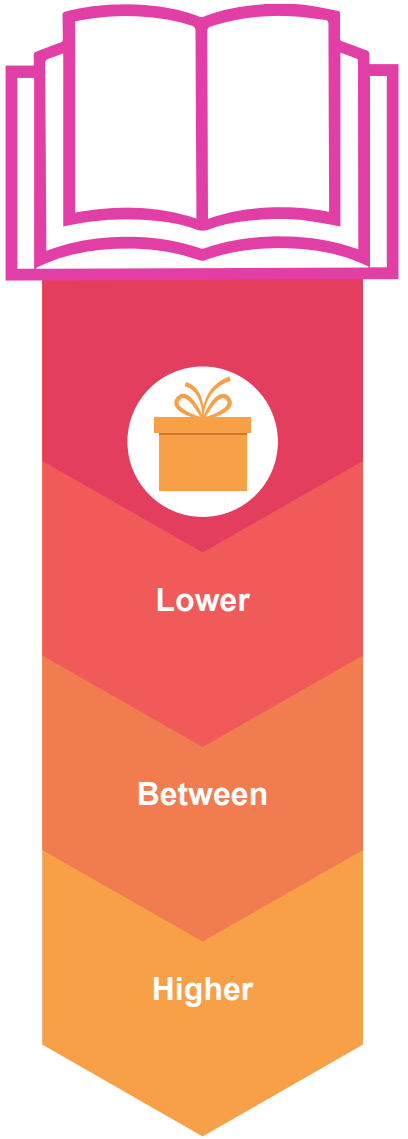


Three way Partitioning of an Array around a Given Range in Python programming language

- All the elements less than low value, should come first.
- Elements between the low and high value come in middle.
- All elements greater than high should come at the last.

Basic Idea


Three Way Partitioning of Array
Given Array
1 17 22 16 13 5 43 18 3 10
Low value : 14
High value : 20
Partition the array under the given condition :
All elements smaller than low value come first.
All elements in range low value to high value come next.
All elements greater than high value appear in the end
Array after Partitioning
1 5 3 13 10 17 16 18 22 43
Elements less than 14
Elements between 14 and 20
Elements more than 20


Lower
Between
Higher

Python Program

```
def partition(arr, l, h):
    lm = []
    mm = []
    hm = []
    for i in arr:
        if i < l:
            lm.append(i)
        elif i > h:
            hm.append(i)
        else:
            mm.append(i)
    return lm + mm + hm
array = [1, 17, 22, 16, 13, 5, 43, 18, 3, 10]
lowVal = 14
highVal = 20
print("After Partitioning :", partition(array, lowVal, highVal))
```

Output

After Partitioning :

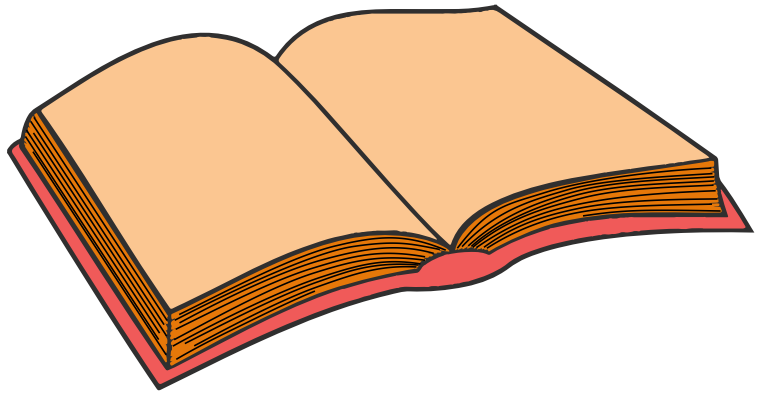
[1, 13, 5, 3, 10, 17, 16, 18, 22, 43]

Move all the negative elements to one side of the array using Python

Method 1: Move to left

All the negative elements will move to the left side of array as negative element are small.

Array after moving all the elements to left:
[-6, -5, -3, -1, 1, 3, 3, 4, 7]



```
def find(arr):
    # sort array
    arr.sort()
    # print array
    print("Array after moving all the elements to left:", arr)
array = [1, 3, -1, 4, -3, -5, -6, 3, 7]
# call function
find(array)
```

Contd….

Method 2: Move to right

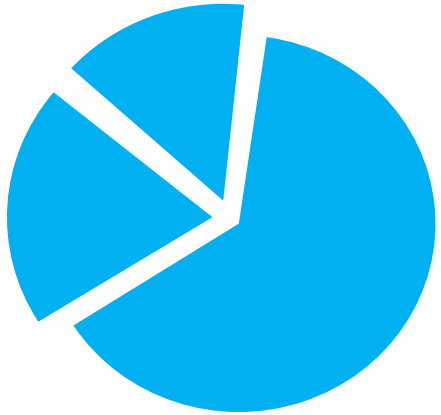
All the negative elements will move to the right side of array.

Array after moving all the elements to left:
[7, 4, 3, 3, 1, -1, -3, -5, -6]

```
def find(arr):
    # sort array
    arr.sort(reverse=True)
    # print array
    print("Array after moving all the elements to right:", arr)
array = [1, 3, -1, 4, -3, -5, -6, 3, 7]
# call function
find(array)
```

Contd….

```
def find(arr):
    ans = []
    for i in arr:
        if i < 0:
            ans.append(i)
        if 0 in arr:
            ans.append(0)
    for i in arr:
        if i > 0:
            ans.append(i)
    print("Array after moving all the elements:", ans)
array = [1, 3, -1, 4, -3, -5, -6, 3, 7]
# call function
find(array)
```



Method 3

Array after moving all the elements:
[-1, -3, -5, -6, 1, 3, 4, 3, 7]

```
def find(arr):
    ans = []
    for i in arr:
        if i < 0:
            ans.append(i)
        if 0 in arr:
            ans.append(0)
    for i in arr:
        if i > 0:
            ans.append(i)
    ans = ans[::-1]
    print("Array after moving all the elements:", ans)
array = [1, 3, -1, 4, -3, -5, -6, 3, 7]
# call function
find(array)
```

Contd….

Method 4

Array after moving all the elements:
[7, 3, 4, 3, 1, -6, -5, -3, -1]





Finding the Kth smallest element

01.02.2023

Find the "Kth" max and min element of an array

Find kth max

Step 1: Sort the array in descending order using predefined function sort
Step 2: print array[k-1] after sorting the array in descending order



Find kth min

Step 1: Sort the array in ascending order using predefined function sort
Step 2: print array[k-1] after sorting the array in ascending order

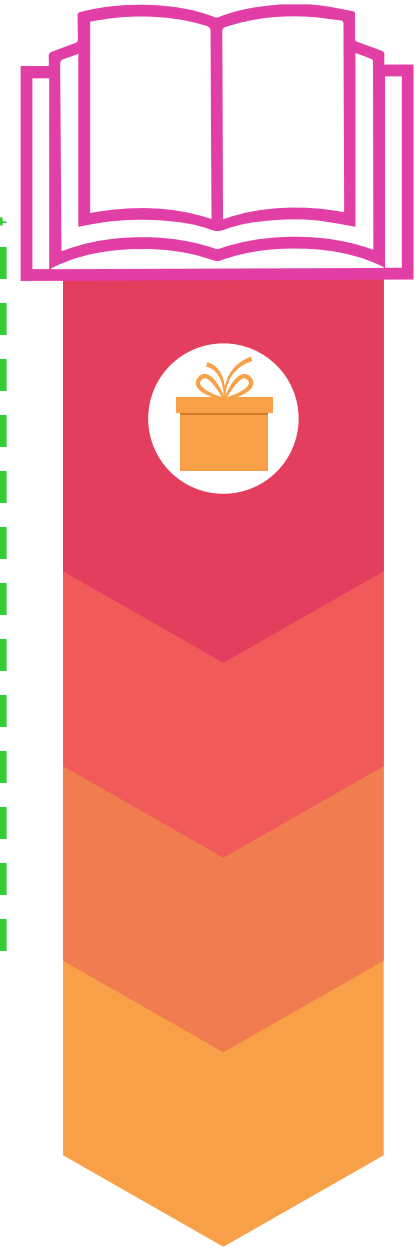
Program

```
def findkmax(k, array):
    # sort the array in descending order
    array.sort(reverse=True)
    # Since indexing start from 0, hence if user want first number-index=0
    print("K th maximum element is: ", array[k - 1])
def findkmin(k, array):
    # sort the array in ascending order
    array.sort()
```



Contd…

```
# Since indexing start from 0, hence if user want first number-index=0
    print("K th minimum element is: ", array[k - 1])
array = [1, 7, 6, 8, 9, 2, 4, 5, 3, 0]
k = 2
# call function findkmax
findkmax(k, array)
# call function findkmin
findkmin(k, array)
```



Output:

K-th maximum element is: 8

K-th minimum element is: 1

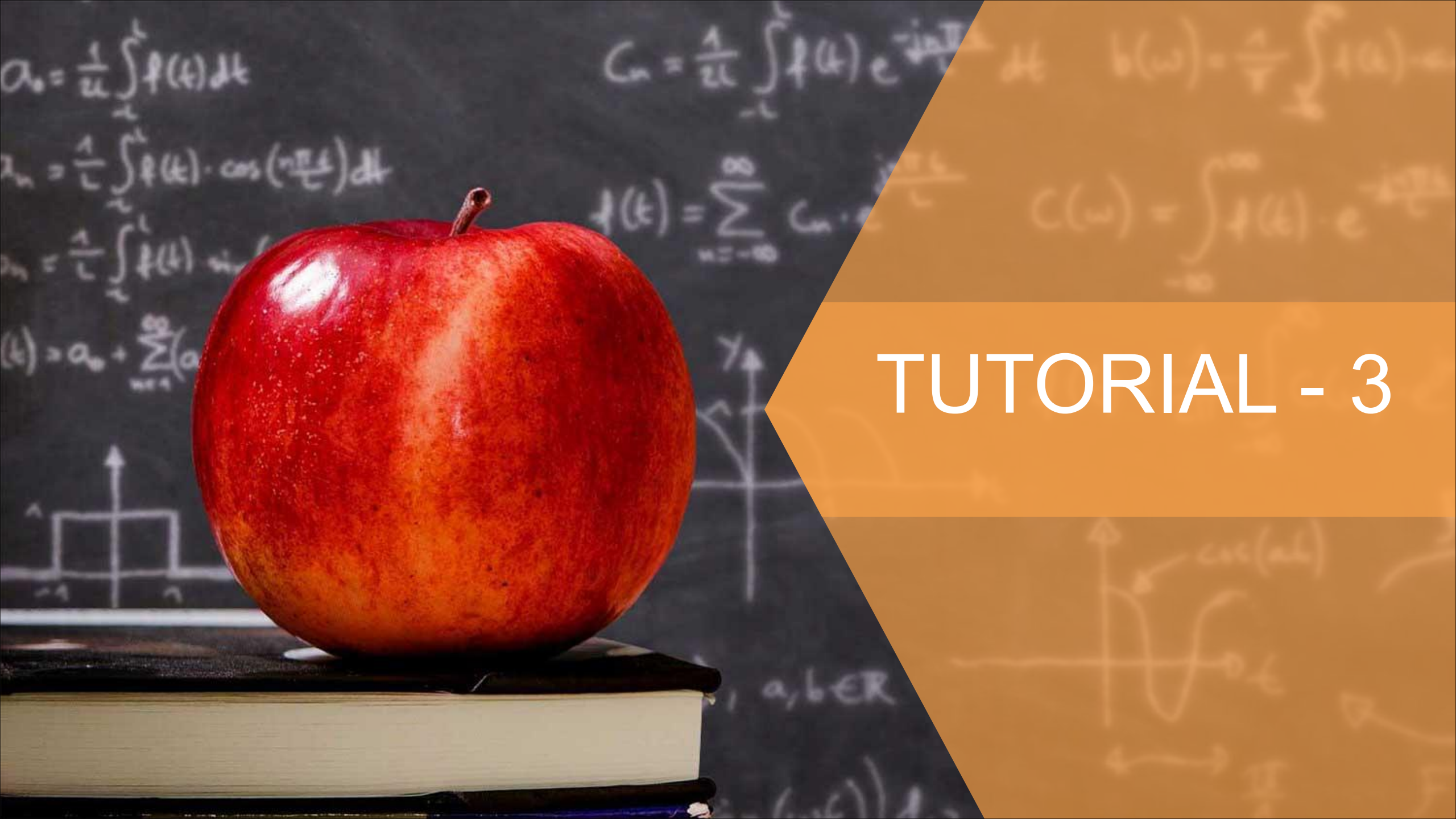
Program

```
def find(k, arr, l):
    # sort the array in descending order
    arr.sort(reverse=True)
    # Since indexing start from 0, hence if user want first number-index=0
    print("K th maximum element is: ", arr[k - 1])
    # Since the array is sort in reverse order for minimum - find from end
    print("K th minimum element is: ", array[l - k])
array = [1, 7, 6, 8, 9, 2, 4, 5, 3, 0]
k = 2
# call function
find(k, array, len(array))
```





THANK YOU



TUTORIAL - 3

Tutorial -3 Questions

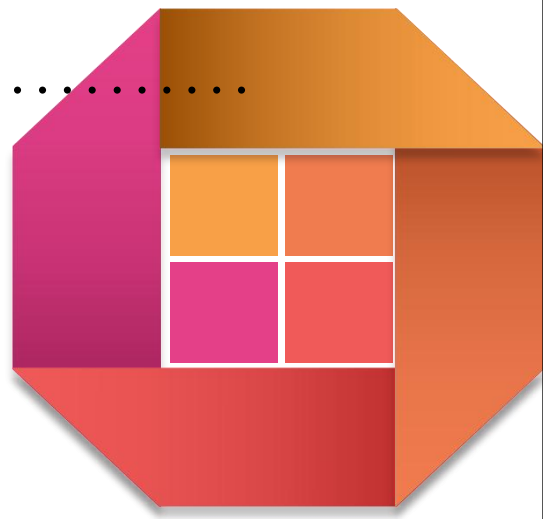
1. Write a Python program to convert decimal into other number systems (Binary, octal and Hexadecimal for your Date of Birth, Month and year. [Eg: If your DOB: 25.08.2005, Find for 25, 08, 2005)

2. Find the Output for the following

(i) If Dict = {1:2, 3:4, 4:11, 5:6, 7:8}, then print (Dict [Dict [3]]) will print

a. 2 b. 8 c. 11 d. 6

(ii) If List = [1,2,4,5,6,7,8,9,10] , then print List[8:4:-1] will give ……



Contd...

(iii) Dict = {"India": "New Delhi", "Nepal": "Kathmandu"}

Dict1 = {"USA": "Washington DC"}

Dict.update (Dict1)

Print (Dict)

(iv) Tup = (1, 'abc')

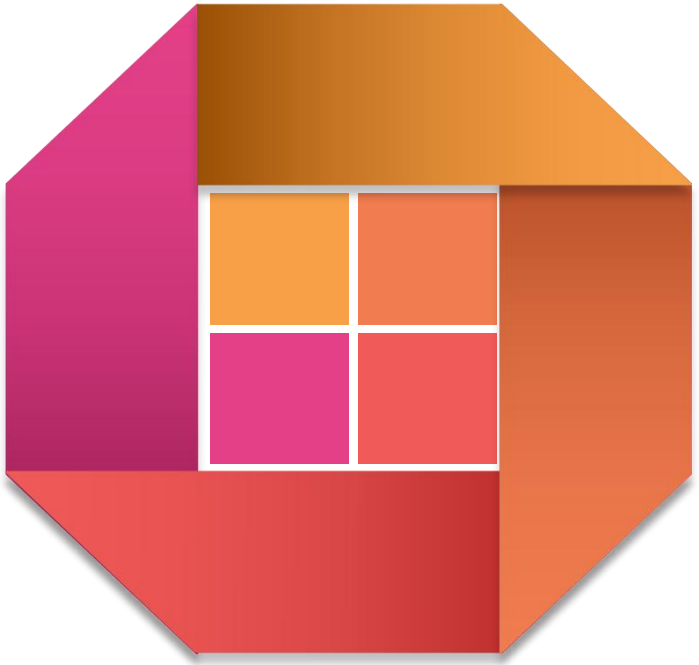
List = [1, 'abc']

print (Tup == List)

print (Tup == tuple (List))

print (list (Tup) == List)

print (1,2) + (3,4)



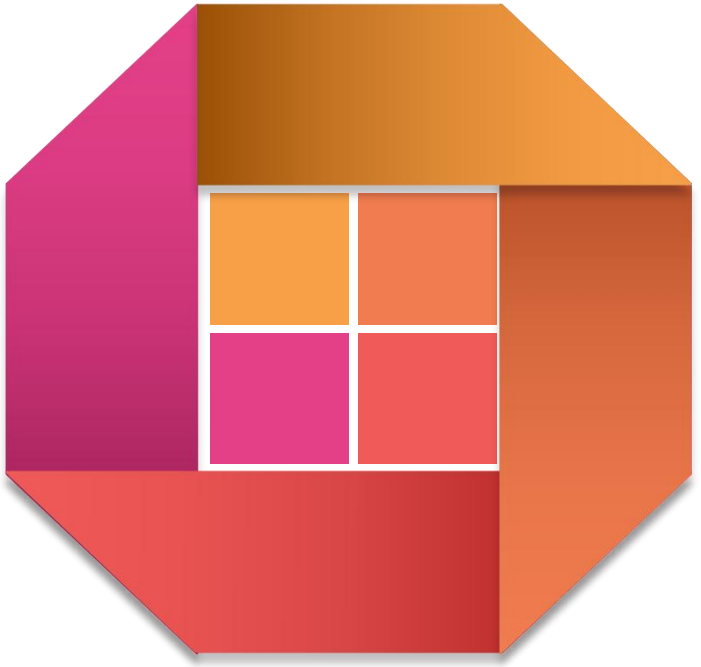
Contd...

(v) Tup = (2,4,6)

```
Add_Tup = Tup + Tup
Print (Add_Tup)
Mul_Tup = Tup*3
Print (Mul_Tup)
```

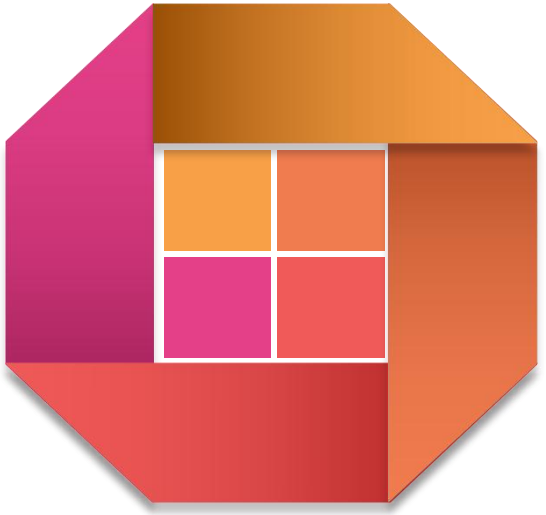
(vi)

```
def reverseList(A):
    print( A[::-1])
# Driver function to test above function
A = [1, 2, 3, 4, 5, 6]
print(A)
print("Reversed list is")
reverseList(A)
```



Contd...

3. Write a Program that has a dictionary of your friends name (as Keys) and their birthdays. Print the item in the dictionary in a sorted order. Prompt the user to enter a name and check if it is present in the dictionary. If the name does not exist, then ask the user to enter DOB. Add the details in the dictionary.



Contd...

4. Find the error

(i) x = {1,2,3,4,5}

x.add ([6,7,8])

print (x)

(ii) list =[(1,2), [3,4], '56', 78, 9.0]

list.remove ('abc')

(iii) Dict { }

Print (Dict [0])

(iv) Tup1 = (9,8,7,6,5)

Tup2 = (1,2,3,4,5)

Print (Tup1 – Tup2)

